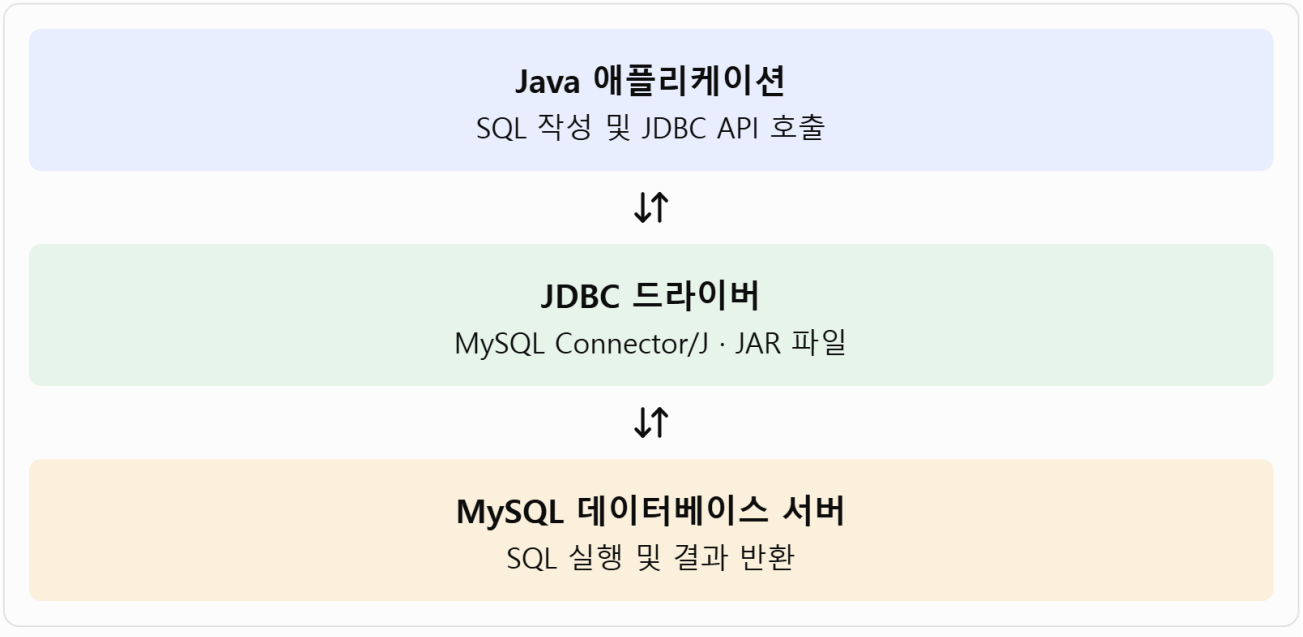
JDBC

inky4832@daum.net

1.1 JDBC

JDBC (Java Database Connectivity)

- 개념
  - Java 프로그램에서 데이터베이스에 접속하고 SQL을 실행하기 위한 표준 API.
  - 거의 대부분의 표준 API가 인터페이스(interface)로 제공됨.
  - 제공된 인터페이스(interface)를 실제 vendor에서 클래스로 구현하여 JAR 로 제공됨.



1.2 JDBC 주요 API

주요 클래스 및 인터페이스

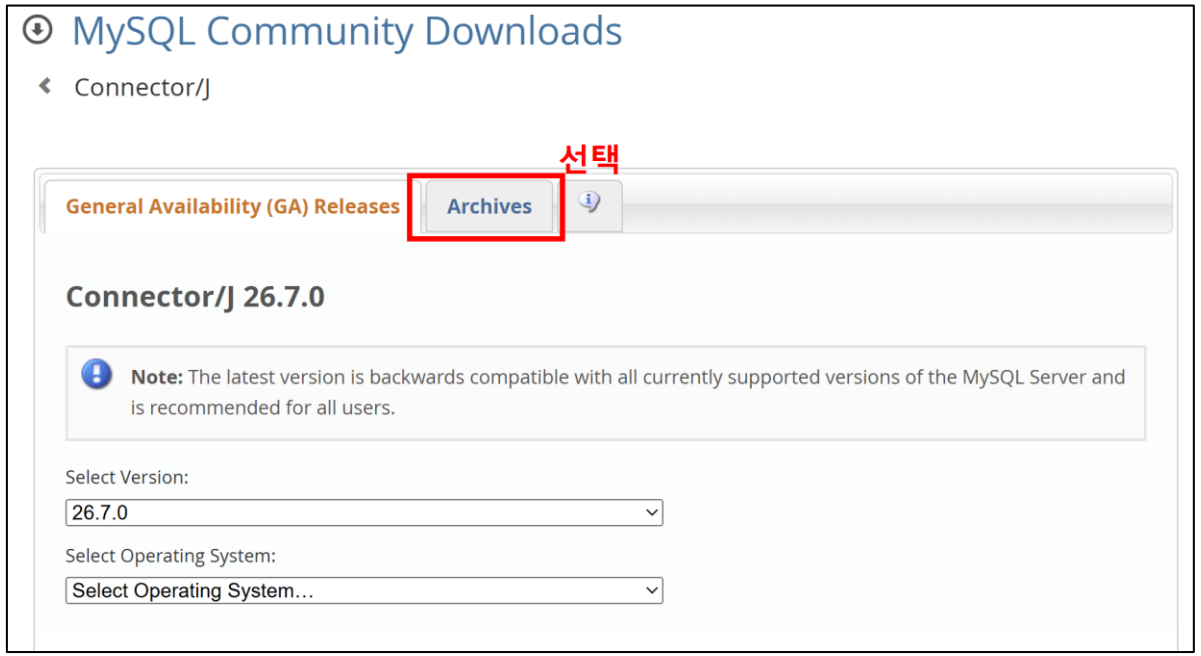
https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/package-summary.html

클래스/인터페이스	역할
DriverManager	데이터베이스 연결 관리
Connection	데이터베이스 연결 객체
PreparedStatement	SQL 실행 객체
ResultSet	SELECT 조회 결과 처리 객체
SQLException	SQL 실행 및 DB 연결 관련 예외 객체

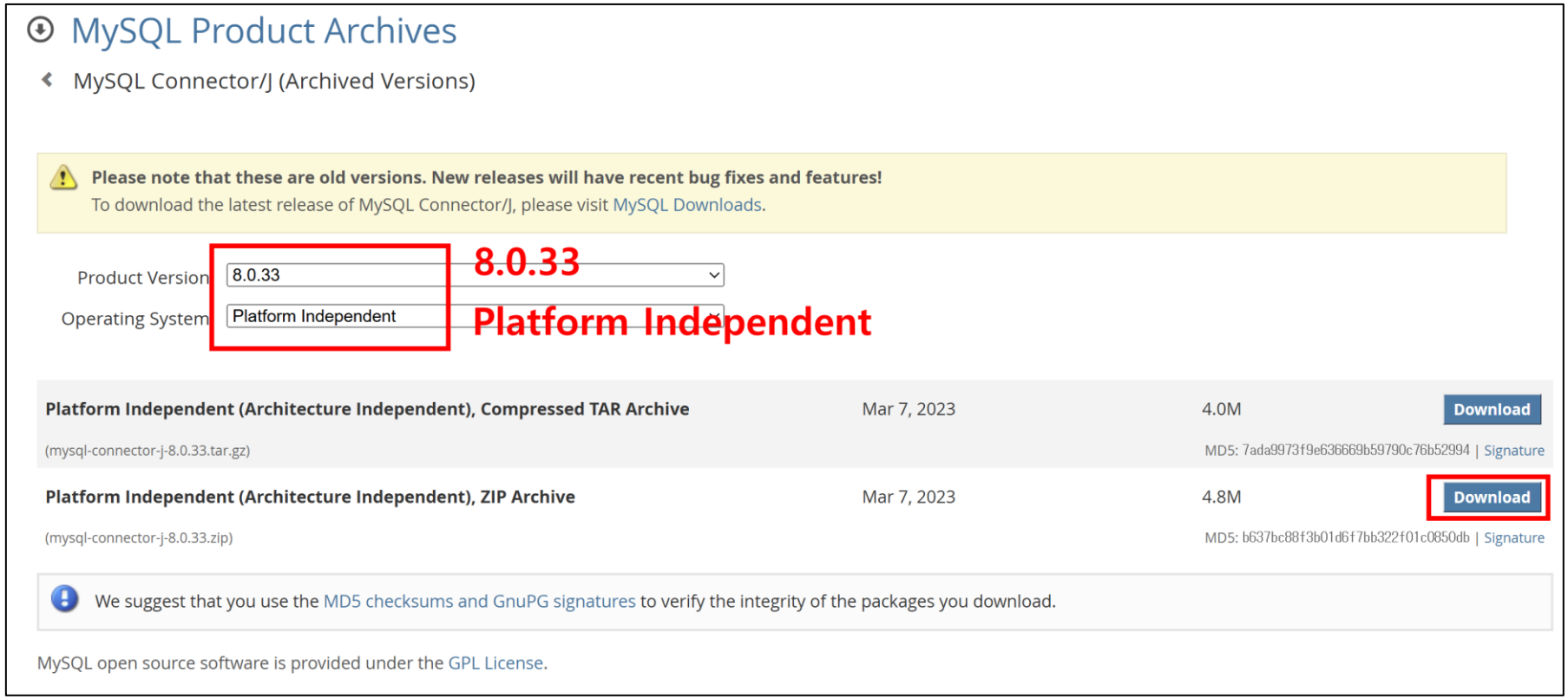
1.3 JDBC 드라이버 설정

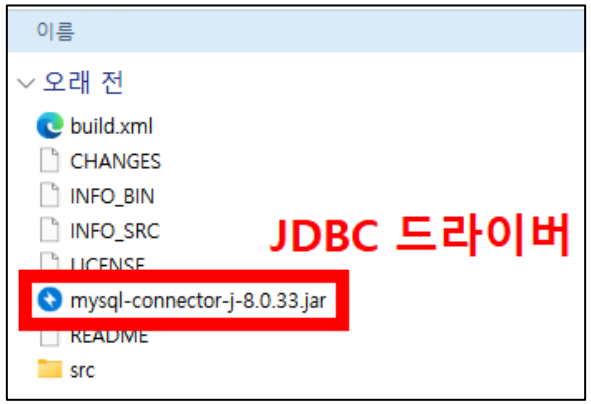
1. JDBC 드라이버 다운로드

- https://dev.mysql.com/downloads/connector/j/



1.3 JDBC 드라이버 설정

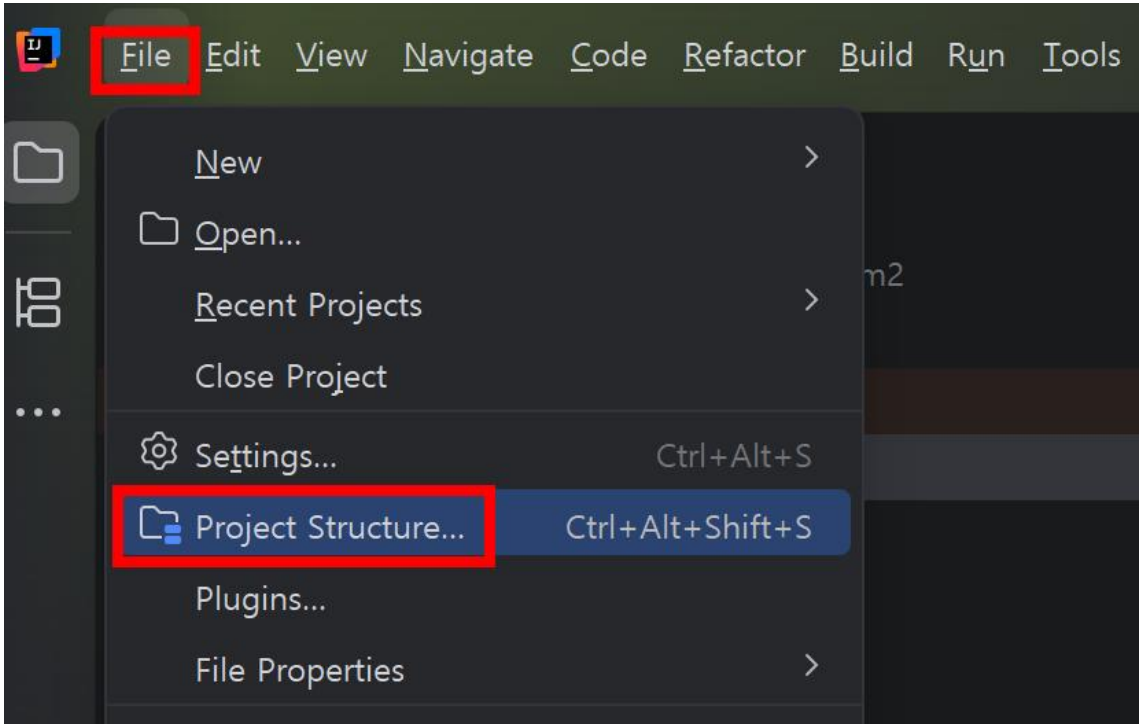




1.3 JDBC 드라이버 설정

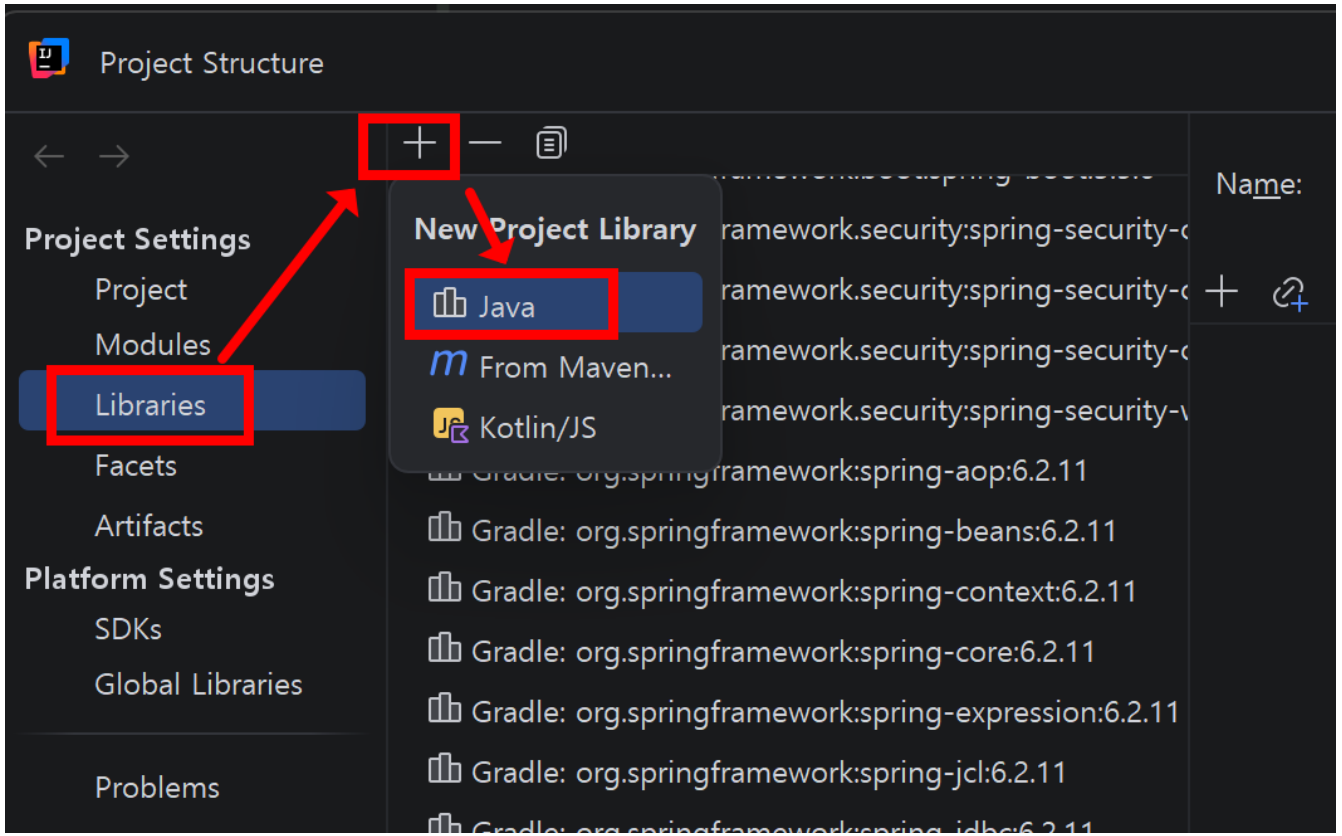
2. Intellij 에서 MySQL 드라이버 설정하기

File > Project Structures.. 선택



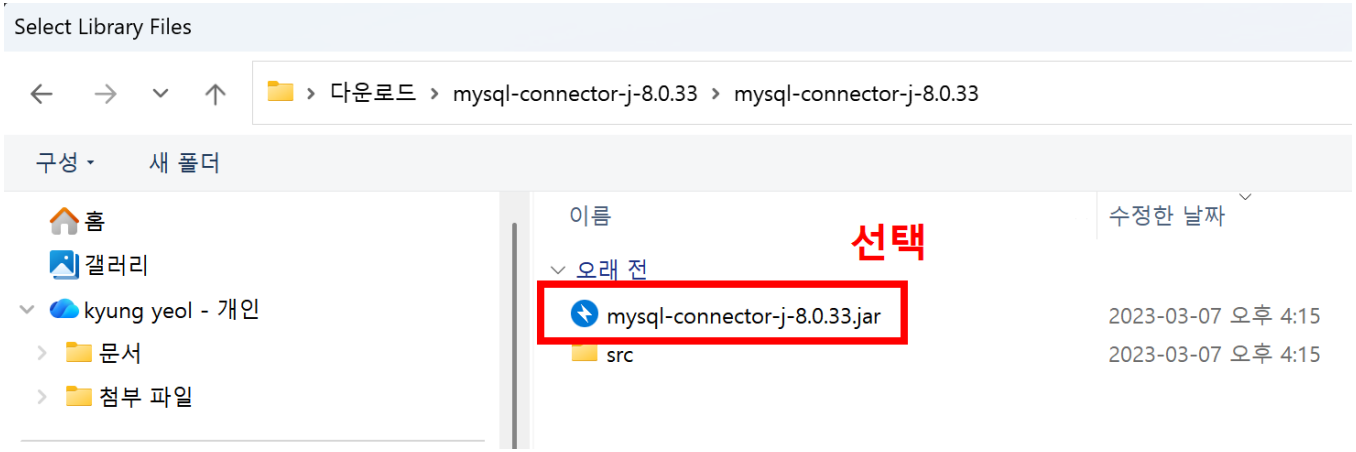
1.3 JDBC 드라이버 설정

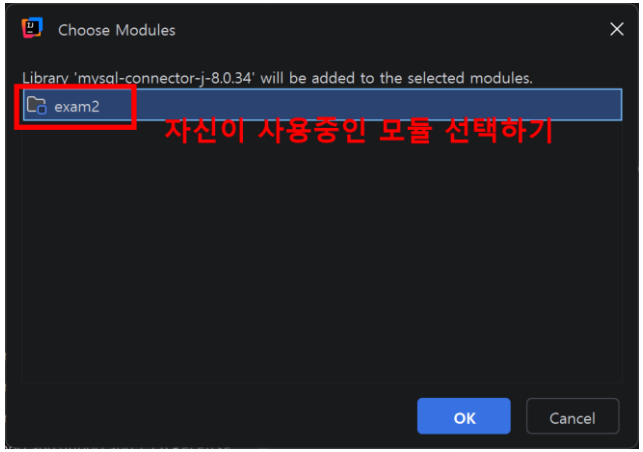
Libraries > + 선택 > Java 선택 > MySQL 드라이버 선택하기

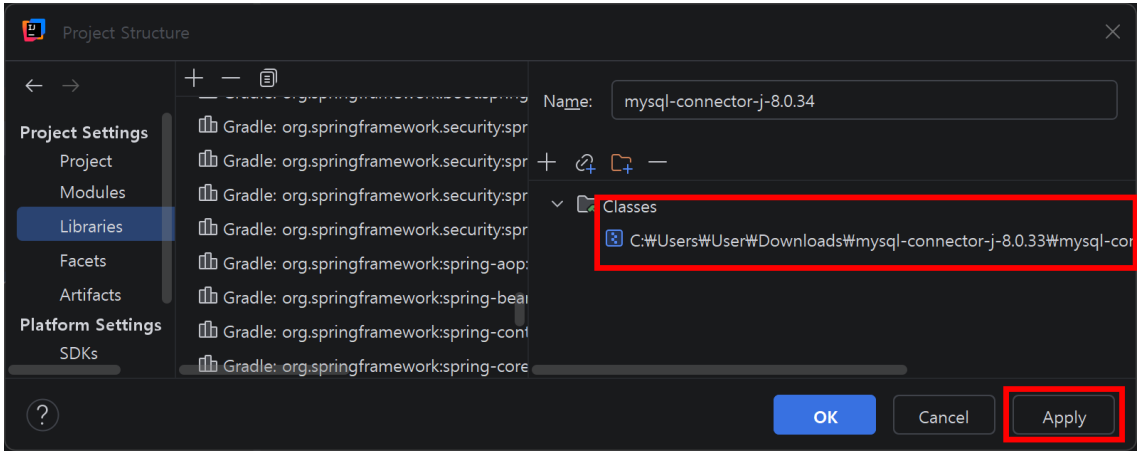


1.3 JDBC 드라이버 설정

MySQL 드라이버 선택하기 > Module 선택 > Apply 선택 > OK 선택

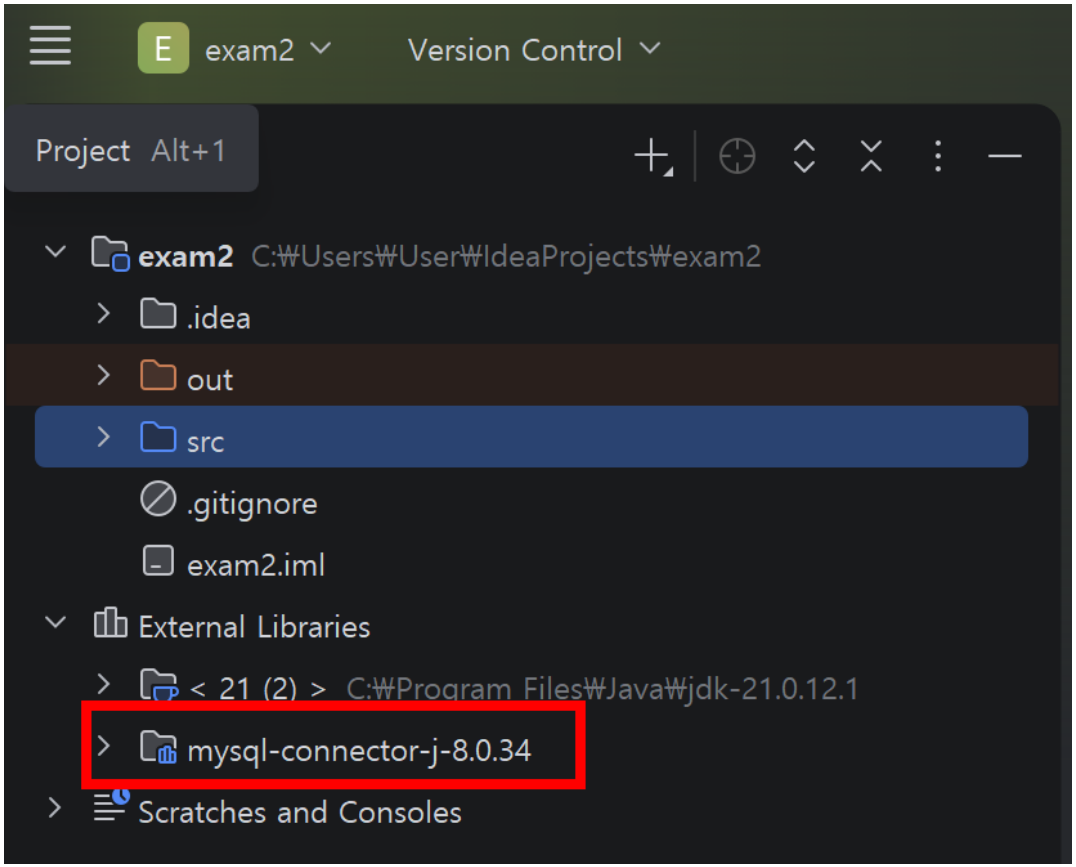






1.3 JDBC 드라이버 설정

3. External Libraries 에서 JAR 확인



2.1 JDBC 연동

1. MySQL 접속 정보 4가지 설정

```
String driver = "com.mysql.cj.jdbc.Driver";
String url = "jdbc:mysql://localhost:3306/testdb";
String userid ="root";
String passwd = "1234";
```

2. 드라이버 로딩

```
try{
    Class.forName(driver);
}catch(ClassNotFoundException e){}
```

2.1 JDBC 연동

3. Connection 연결

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

try{
    Connection con = DriverManager.getConnection(url, userid, passwd);
}catch(SQLException e){}
```

4. SQL 문 작성

```
// SELECT 문
String sql = "select deptno, dname, loc from dept";

// DML 문
String sql = "insert into dept(deptno,dname,loc) values(50,'인사','서울')";

String sql = "insert into dept(deptno,dname,loc) values(?,?,?)";
```

2.1 JDBC 연동

5. PreparedStatement 생성 (SQL문 전달 객체)

```
import java.sql.PreparedStatment;
import java.sql.SQLException;

try{
    PreparedStatment pstmt = con.prepareStatement(sql);
}catch(SQLException e){}
```

6. SQL 문 전송 및 결과 받기

```
가. DML ( insert, delete, update )

try{
  int n = pstmt.executeUpdate(); // n에 레코드 변경 개수가 저장
}catch(SQLException e){}
```

2.1 JDBC 연동

```
나. DQL ( select )

import java.sql.ResultSet;

try{
  // select deptno,dname,loc from dept

  ResultSet rs = pstmt.executeQuery(); // rs에 실행결과가 저장됨
  while(rs.next()){  // 행선택
    // 열선택
    int deptno = rs.getInt("deptno");      // rs.getInt(1);
    String dname = rs.getString("dname"); // rs.getString(2);
    String loc = rs.getString("loc");      // rs.getString(3);
  }

}catch(SQLException e){}
```

2.1 JDBC 연동

7. 자원 반납

```
- 사용순서
Connection con;
PreparedStatment pstmt;
ResultSet rs;

- close 순서는 사용순서 역순으로 해야 됨.
try{
  if(rs != null) rs.close();
  if(rs != null) pstmt.close();
  if(rs != null) con.close();
}catch(SQLException e){}
```

3.1 트랜잭션 처리

트랜잭션 처리

- JDBC에서의 DML은 기본적으로 auto commit 임.

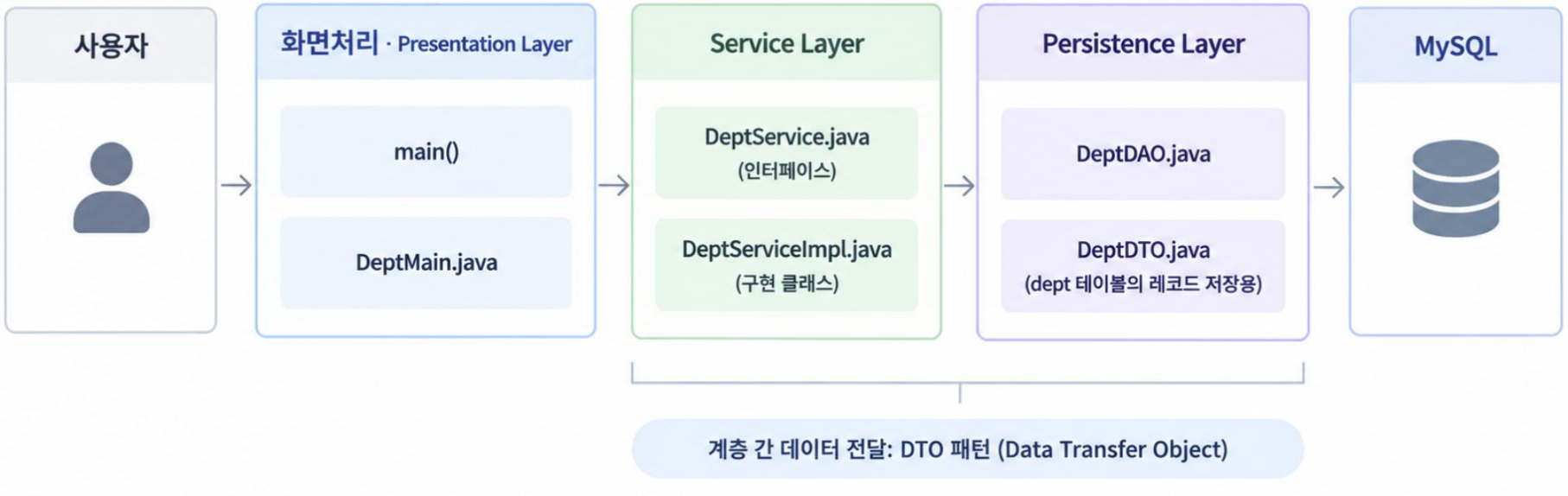
```
try{
   con.setAutoCommit(false);  // 기본값은 true


    //insert문
    //update문

    con.commit();
}catch(SQLException e){
    con.rollback();
}
```

4.1 Layer로 나누기

계층형 아키텍처 (Layered Architecture)



수고하셨습니다.